

 BLOCK 1 — DSA CONCEPT

 Queue (Real-world usage)

 API server works like this:

Incoming requests

↓

Queue (FIFO)

↓

Processed one by one

 Example (your system)

User1 → "BST"

User2 → "Heap"

User3 → "AVL"

 Server handles:

User1 → first

User2 → next

User3 → next

 FIFO = Queue

 Extra (important insight)

Your current system:

while True → input()

 Only 1 user at a time 

After FastAPI: Multiple users → API → handled asynchronously 

BLOCK 2 — AI / SYSTEM CONCEPT

✅ What is FastAPI?

👉 Turns your code into:

function → HTTP endpoint

✅ Your current system

Python script ❌

✅ After Day 3

http://localhost:8000/ask ✅

✅ Flow

User → HTTP request → FastAPI → Your RAG → JSON response

BLOCK 3 — BUILD (FULL CODE FOR YOU)

✅ Step 1 — Install

pip install fastapi uvicorn

✅ Step 2 — Create main.py

👉 Use THIS (adapted to YOUR project 👉)

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

```
from your_file import ChatMemory # 📄 change to your filename
```

```
# ✅ create app
```

```
app = FastAPI()
```

```
# ✅ initialize your RAG system

memory = ChatMemory()

memory.prepare_document(path="final_clean_dsa.md", # your cleaned file ✅
    size=500, overlap=100)

# ✅ request schema

class ChatRequest(BaseModel):
    message: str

# ✅ response schema

class ChatResponse(BaseModel):
    reply: str

# ✅ main endpoint

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(req: ChatRequest):
    reply = memory.chat(req.message)
    return ChatResponse(reply=reply)
```

✅ Step 3 — Run server

```
uvicorn main:app --reload
```

✅ Step 4 — Open UI

👉 Open browser: <http://127.0.0.1:8000/docs>

✅ You will see:

- Swagger UI ✅
 - Test interface ✅
 - API working ✅
-

✅ Step 5 — Test

Try:

```
{  
  "message": "What is AVL tree?"  
}
```

✅ Response:

```
{  
  "reply": "AVL tree is a self-balancing binary search tree..."  
}
```

🔥 ✅ WHAT YOU JUST BUILT

Before	After
CLI chatbot	❌
API chatbot	✅
Single user	❌
Multi-user ready	✅

🧠 BLOCK 4 — REAL ENGINEERING THINKING

✅ 1. What happens if 2 users call at same time?

👉 Current problem: shared ChatMemory ❌

👉 Future fix (Day 5): session-based memory ✅

✅ 2. What if file not loaded?

👉 Add safeguard:

if not memory.loader.chunks:

```
    return {"reply": "Document not loaded"}
```

✅ 3. Latency?

Your pipeline: Embedding + BM25 + Chroma + LLM

👉 Slight delay (~2–5 sec) ✅ normal

✅ ✅ VERIFICATION CHECKLIST

Make sure:

- ✅ server runs without error
 - ✅ /docs opens
 - ✅ POST /chat works
 - ✅ returns JSON
 - ✅ answers from your document
-

🚀 WHERE YOU ARE

- ✅ Day 1 → BM25
- ✅ Day 2 → Hybrid search
- ✅ Day 3 → API ✅

👉 You are now building production-level system 🔥